

Design Doc: Catálogo Offline Global e Sincronização Delta de Escaladas

Aresta Climb • Arquitetura de Alto Desempenho e Dados Offline

STATUS

Proposto

AUTOR

Renato / Core Team

DATA & VERSÃO

Outubro 2026 • v1.0

MÓDULOS

aresta_app & aresta_db

1. Sumário Executivo

Este documento especifica a nova arquitetura para **busca textual global de escaladas 100% offline** e **resolução instantânea de QR Codes físicos em formato de identificador opaco** (`https://aresta.cc/<nanoid>`), eliminando completamente o consumo de memória RAM na inicialização do aplicativo e viabilizando atualizações incrementais de dados de apenas **~1.5 KB** sobre redes celulares precárias (3G/4G).

Os Três Pilares da Solução:

- Tabela de Roteamento Offline (SQLite):** Um banco embarcado único (`catalogo.db`, ~250 KB comprimido) consultado sob demanda do disco (0 ms no boot, 0 MB na RAM).
- Simbiose com o Índice na RAM:** Aproveitamento dos metadados já residentes no `indice.binarypb` para eliminar duplicidade de strings no banco.
- Sincronização Delta na Borda (Cloudflare Worker):** Entregas de atualizações ultra-leves na Edge com cache imutável e fallback automático para snapshot.

2. Contexto e Motivação

2.1 O Problema da Abordagem Ingênua (Carregar Tudo no Boot)

Atualmente, vias e setores residem dentro de cada arquivo `compilado.binarypb`. Carregar todos os croquis na inicialização geraria uma falha crítica de performance:

- Overhead de Heap Dart 64-bit:** Protobufs em memória expandem entre 5x a 10x. Para 500 picos, o app alocaria **150 MB a 300 MB de RAM**, provocando encerramentos arbitrários pelo sistema operacional (Low Memory Killer).
- Congelamento da Splash Screen:** A desserialização síncrona bloquearia a UI thread por **2 a 6 segundos** no cold start.

2.2 O Desafio dos QR Codes Opacos

A nova geração de placas físicas e links compartilháveis adota identificadores opacos:

`https://aresta.cc/xkwjf8dk3ljkl3` (NanoID Base62 de 14 caracteres). Como a URL não revela em qual arquivo o pico reside, o aplicativo necessita de um índice reverso leve para encontrar o morro instantaneamente no modo offline.

3. Arquitetura Proposta em Duas Camadas

ARQUITETURA EM DUAS CAMADAS

1. CAMADA DE ROTEAMENTO E BUSCA (GLOBAL)

Arquivo: 'catalogo.db' (SQLite local, ~250 KB sincronizado)

Responsabilidade:

- Resolver QR Codes instantaneamente (NanoID → PicoId, SetorId)
- Alimentar a Busca Global por qualquer via, grau e setor (FTS5)
- 100% offline, 0 ms de overhead no boot, 0 MB na RAM.

Escaneou QR Code / Clicou na Busca
NanoID "xkwjf8dk3ljkl3" → Pico: "pedra_grande"

2. CAMADA DE CONTEÚDO GRÁFICO (SOB DEMANDA)

Arquivos: '<picoId>/compilado.binarypb' e Imagens

Responsabilidade:

- Traçados vetoriais, nós de desenho, linhas, polígonos, fotos.
- Baixado sob demanda (online) OU sincronizado pelo usuário (offline).
- Carregado na memória RAM APENAS quando a tela daquele pico específico abre!

3.1 Simbiose entre `Índice` (RAM) e `Catalogo` (Disco)

O aplicativo já carrega o `indice.binarypb` (~11 KB) na memória RAM na abertura do app. O índice já contém a lista com todos os picos cadastrados. Logo, **o banco SQLite não precisa duplicar nomes de picos, municípios ou caminhos de download**. O SQLite armazena apenas o `croqui_uid`. Ao encontrar o registro, o app resolve o nome do pico na RAM em tempo $O(1)$ sem custo de I/O.

4. Especificação Técnica do Banco de Dados

4.1 Schema SQL (`catalogo\_aresta`)

```
CREATE TABLE catalogo_aresta (
  uid TEXT PRIMARY KEY,           -- NanoID Base62 de 14 caracteres
  croqui_uid TEXT NOT NULL,       -- UID do croqui pai (conecta ao indice.binarypb)
  tipo_id INTEGER NOT NULL,       -- 1: grupo, 2: setor, 3: escalada, 4: botao
  -- Hierarquia Textual (para busca instantânea sem JOINS)
  nome_grupo TEXT,               -- Preenchido se pertencer a um grupo
  nome_setor TEXT,               -- Preenchido para setores e vias
  nome_escalada TEXT,            -- Preenchido APENAS para escaladas
  -- Metadados de Escalada
  grau TEXT,                     -- Ex: '7a', 'V5', 'E2'
  tipo_escalada INTEGER          -- 1: esportiva, 2: boulder, 3: movel, 4: multienfiada, 5: high
);

CREATE INDEX idx_catalogo_croqui ON catalogo_aresta(croqui_uid);
CREATE INDEX idx_catalogo_tipo ON catalogo_aresta(tipo_id);
```

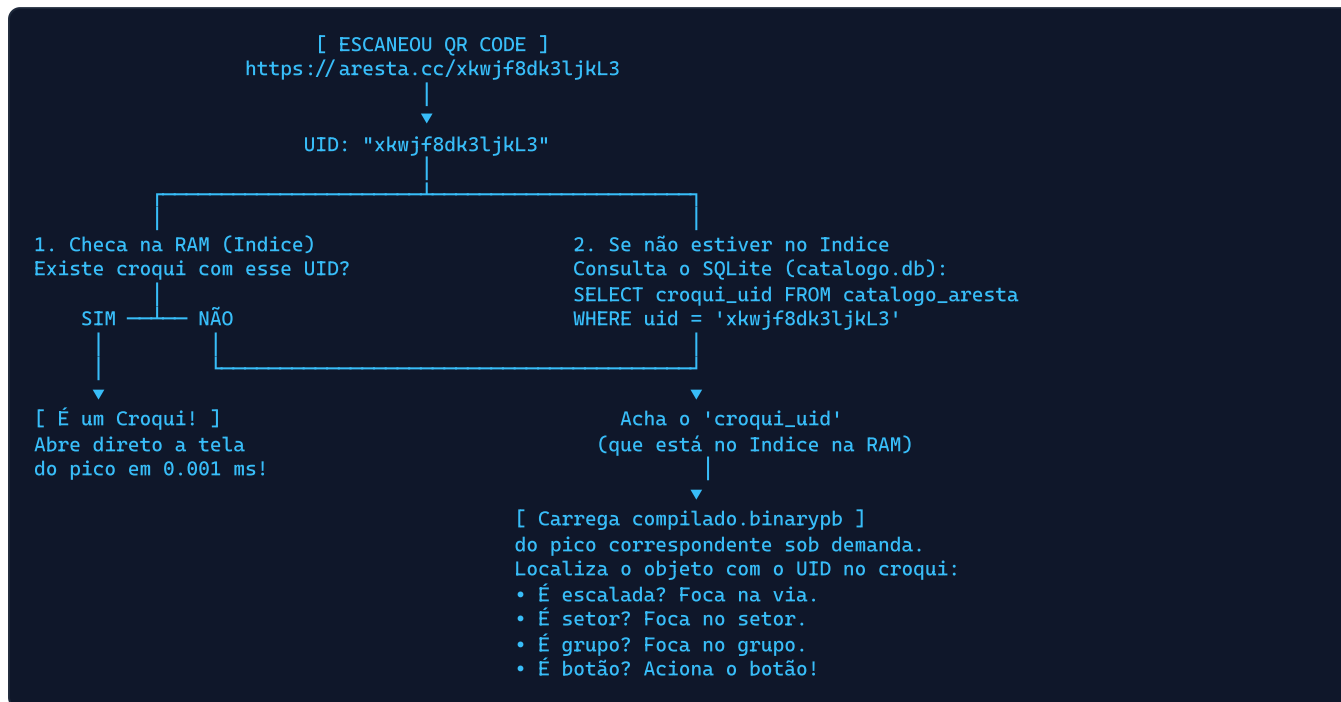
4.2 Densidade e Tamanho Físico

No SQLite, colunas com valor `NULL` gastam apenas 1 byte de controle no cabeçalho do registro, e inteiros pequenos (1 a 5) gastam 1 byte.

ENTIDADE	OCUPAÇÃO MÉDIA POR LINHA	VOLUME PARA 20.000 REGISTROS	VOLUME COMPRIMIDO (GZIP -9)
Escalada (Via/Boulder)	~45 bytes	~900 KB	~220 KB a 260 KB
Setor / Grupo	~38 bytes	Incluso	Incluso
Botão (PIX/Manutenção)	~34 bytes	Incluso	Incluso

5. Fluxos de Execução

5.1 Resolução de QR Codes e Universal Links



5.2 Busca Global de Escaladas

A barra de busca pesquisa na tabela local usando SQL direto:

```
SELECT uid, croqui_uid, tipo_id, nome_grupo, nome_setor, nome_escalada, grau, tipo_escalada
FROM catalogo_aresta
WHERE nome_escalada LIKE '%fissura%' OR nome_setor LIKE '%fissura%'
LIMIT 30;
```

A UI formata o resultado em tempo real consumindo o `Índice` da RAM: `Fissura do Totem • 7a • Setor Central • Pedra Grande (MG)`.

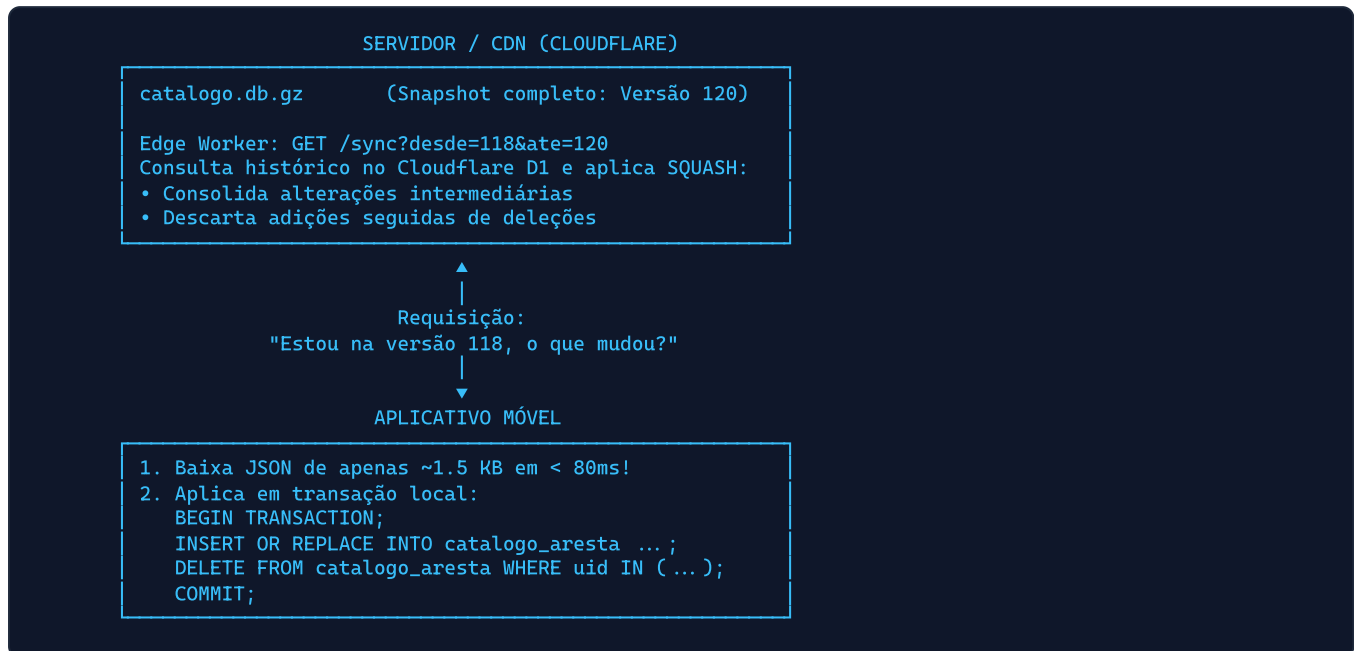
6. Pipeline de Compilação & CI/CD (`deploy_generated.py`)

O script `deploy_generated.py` no repositório `aresta_db` incorpora um novo passo após a compilação dos protobufs:

- **Deploy Total:** Reconstrói o `catalogo.db` em uma única transação SQLite (**50 a 100 ms** para todo o catálogo do Brasil).
- **Deploy Seletivo (`target_paths`):** Executa limpeza cirúrgica e reinserção apenas do croqui modificado:
`DELETE FROM catalogo_aresta WHERE croqui_uid = :id; (< 5 ms).`
- **Pre-bundling:** Durante o build do app, o arquivo `catalogo.db` é embutido em `assets/preload/catalogo.db`, garantindo funcionamento offline logo após a instalação na loja.

7. Sincronização Delta Remota com Cloudflare Edge Worker

Para sincronizações no dia a dia, um Cloudflare Worker na borda entrega apenas os deltas das versões pendentes:



7.1 As Três Garantias de Confiabilidade da Sincronização

- **URLs Determinísticas e Cache Imutável:** A versão remota é declarada no `indice.binarypb` (`versao_catalogo = 120`). A chamada do app solicita intervalo fechado (`desde=118&ate=120`), permitindo cache infinito na CDN global.
- **Cálculo Determinístico no GitHub Actions:** O CI calcula os deltas automaticamente via SQL `EXCEPT` comparando a nova compilação com a versão anterior baixada da CDN.
- **Snapshot de Segurança (Fallback):** Se a defasagem do cliente for superior a 15 versões ou o payload passar de 200 KB, o sistema simplesmente baixa o `catalogo.db.gz` completo, zerando o risco de estados corrompidos.

8. Quadro Comparativo de Performance

CRITÉRIO	ABORDAGEM INGÊNUA (CARREGAR PROTOBUFS)	ABORDAGEM PROPOSTA (SQLITE + DELTA EDGE)	GANHOS PRÁTICOS
Consumo de RAM no Boot	150 MB a 300 MB	0 MB (carregamento sob demanda do disco)	-100% de Impacto
Tempo de Abertura (Cold Start)	2 a 6 segundos de travamento na splash	0 ms de impacto na inicialização	Boot Instantâneo
Resolução de QR Code Offline	Inviável sem carregar todos os croquis	< 1 ms por busca B-Tree em disco	Resolvido em O(1)
Volume de Dados por Update	~250 KB (ou dezenas de MBs de croquis)	~1.5 KB a 3 KB via Edge Worker	-99% de Tráfego
Comportamento com Sinal Fraco (3G)	Timeout e falha de download	Sincronização concluída em < 80 ms	Excelente UX

9. Roteiro de Implementação

- Fase 1 (aresta_db):** Inclusão de campos NanoID nos schemas e geração de `catalogo.db` / `catalogo.db.gz` no `deploy_generated.py`.
- Fase 2 (aresta_app):** Integração com `sqlite3` / `sqflite`, cópia do asset pré-embutido no `ExtratorAssetsPreload` e unificação da `BuscaGlobal`.
- Fase 3 (aresta_app):** Integração com o leitor de QR Code e App Links (`aresta.cc/<nanoid>`) resolvendo diretamente via banco local.
- Fase 4 (Infraestrutura):** Deploy do Cloudflare Worker com D1 e script de diff com SQL `EXCEPT` no GitHub Actions.